Top 30 Most Asked Coding Questions with Solutions

Essential DSA Preparation for Campus Recruitment

- Curated list of high-frequency interview questions
- Optimized Time & Space Complexities for every problem
- Concise algorithmic strategies for solving
- Spans Arrays, Strings, Lists, Trees, Graphs, and DP
- Perfect for quick reference before coding rounds

Generated by AI Resume Analyzer - Coding Test Practice Module

Date: June 2026

Coding Interview Cheat Sheet

1. Two Sum (Arrays & Hashing)

Complexity: Time: O(N), Space: O(N)

Problem: Find two numbers in an array that add up to a specific target.

Strategy: Use a Hash Map to store the difference (target - nums[i]) and check if it already exists as you iterate.

2. Best Time to Buy & Sell Stock (Arrays)

Complexity: Time: O(N), Space: O(1)

Problem: Find the maximum profit you can achieve by buying and selling a stock.

Strategy: Maintain a minimum price seen so far and calculate profit at each step, updating the max profit.

3. Contains Duplicate (Arrays)

Complexity: Time: O(N), Space: O(N)

Problem: Check if any value appears at least twice in a given array.

Strategy: Insert elements into a Hash Set. If an element already exists in the set, return true.

4. Product of Array Except Self (Arrays)

Complexity: Time: O(N), Space: O(1)

Problem: Return an array where each element is the product of all elements except nums[i].

Strategy: Calculate prefix products in one pass, then multiply by suffix products in a backward pass.

5. Maximum Subarray (Arrays)

Complexity: Time: O(N), Space: O(1)

Problem: Find the contiguous subarray with the largest sum (Kadane's Algorithm).

Strategy: Iterate and maintain current sum. Reset current sum to 0 if it drops below zero, updating global max.

6. Three Sum (3Sum) (Two Pointers)

Complexity: Time: O(N^2), Space: O(1)

Problem: Find all unique triplets in an array that sum to zero.

Strategy: Sort the array, iterate with a fixed element, and use two pointers (left & right) to find pairs.

7. Container With Most Water (Two Pointers)

Complexity: Time: O(N), Space: O(1)

Problem: Find two lines that together with the x-axis forms a container containing the most water.

Strategy: Place pointers at both ends. Move the pointer pointing to the shorter line inward while checking area.

8. Valid Anagram (Strings)

Complexity: Time: O(N), Space: O(1)

Problem: Determine if two strings are anagrams of each other.

Strategy: Count character frequencies using an array of size 26. Compare frequencies for both strings.

9. Group Anagrams (Strings & Hashing)

*Complexity: Time: O(N * K log K), Space: O(N)*

Problem: Group anagrams together from a list of strings.

Strategy: Use a Hash Map where the key is the sorted string and the value is a list of matching anagram strings.

10. Longest Substring Without Repeating Characters (Sliding Window)

Complexity: Time: O(N), Space: O(N)

Problem: Find the length of the longest substring without repeating characters.

Strategy: Use a sliding window with a Hash Set. Move right pointer, if duplicate found, shrink window from left.

11. Valid Parentheses (Stacks)

Complexity: Time: O(N), Space: O(N)

Problem: Check if brackets in a string are closed in the correct order.

Strategy: Push opening brackets to stack. For closing brackets, pop and verify if it matches correct opening type.

12. Reverse Linked List (Linked Lists)

Complexity: Time: O(N), Space: O(1)

Problem: Reverse a singly linked list.

Strategy: Iterative approach: Maintain prev, curr, and next pointers. Change curr->next to prev and step forward.

13. Linked List Cycle (Linked Lists)

Complexity: Time: O(N), Space: O(1)

Problem: Detect if a linked list contains a cycle.

Strategy: Floyd's Tortoise and Hare: Use fast and slow pointers. If they meet, a cycle exists.

14. Merge Two Sorted Lists (Linked Lists)

Complexity: Time: O(N), Space: O(1)

Problem: Merge two sorted linked lists into one sorted list.

Strategy: Use a dummy head node. Compare heads of both lists, attach the smaller node, and advance.

15. Merge Intervals (Sorting)

Complexity: Time: O(N log N), Space: O(N)

Problem: Merge all overlapping intervals.

Strategy: Sort intervals by start times. Iterate and merge overlapping intervals with the last merged interval.

16. Binary Search (Binary Search)

Complexity: Time: O(log N), Space: O(1)

Problem: Find an element index in a sorted array.

Strategy: Use low and high pointers. Compare target with mid element, adjust pointers accordingly.

17. Search in Rotated Sorted Array (Binary Search)

Complexity: Time: O(log N), Space: O(1)

Problem: Search for a target value in a rotated sorted array.

Strategy: Check which half is sorted (left or right). Perform range checks to narrow down search space.

18. Invert Binary Tree (Trees)

Complexity: Time: O(N), Space: O(H)

Problem: Invert a binary tree (mirror image).

Strategy: Recursively swap the left and right children of every node in post-order traversal.

19. Maximum Depth of Binary Tree (Trees)

Complexity: Time: O(N), Space: O(H)

Problem: Find the height of a binary tree.

Strategy: Return 1 + max(depth of left child, depth of right child) recursively.

20. Binary Tree Level Order Traversal (Trees)

Complexity: Time: O(N), Space: O(N)

Problem: Return level-by-level node values.

Strategy: BFS approach: Use a Queue. Process nodes level by level, adding children to the queue.

21. Lowest Common Ancestor of BST (Trees)

Complexity: Time: O(H), Space: O(H)

Problem: Find the LCA of two nodes in a BST.

Strategy: If both nodes are smaller than root, traverse left. If larger, traverse right. Else, root is the LCA.

22. Climbing Stairs (Dynamic Programming)

Complexity: Time: O(N), Space: O(1)

Problem: Find ways to climb N stairs taking 1 or 2 steps.

Strategy: Fibonacci relation: ways(n) = ways(n-1) + ways(n-2). Optimize space using two variables.

23. Coin Change (Dynamic Programming)

*Complexity: Time: O(N * M), Space: O(N)*

Problem: Find minimum coins needed to make a target amount.

Strategy: DP array of size amount+1. dp[i] = min(dp[i], dp[i - coin] + 1) for all coins.

24. Longest Common Subsequence (Dynamic Programming)

*Complexity: Time: O(N * M), Space: O(N * M)*

Problem: Find longest common subsequence between two strings.

Strategy: 2D DP matrix. If characters match, dp[i][j] = 1 + dp[i-1][j-1]. Else, max(dp[i-1][j], dp[i][j-1]).

25. House Robber (Dynamic Programming)

Complexity: Time: O(N), Space: O(1)

Problem: Rob houses in a street without robbing adjacent ones.

Strategy: dp[i] = max(dp[i-1], dp[i-2] + nums[i]). Maintain two variables for space optimization.

26. Number of Islands (Graphs)

*Complexity: Time: O(R * C), Space: O(R * C)*

Problem: Count islands in a 2D binary grid.

Strategy: Iterate grid. When '1' is found, increment island count and run DFS/BFS to mark all connected land as '0'.

27. Clone Graph (Graphs)

Complexity: Time: O(V + E), Space: O(V)

Problem: Create a deep copy of an undirected graph.

Strategy: Use DFS/BFS. Maintain a Hash Map mapping original nodes to their corresponding cloned nodes.

28. Course Schedule (Graphs)

Complexity: Time: O(V + E), Space: O(V + E)

Problem: Detect if you can finish all courses given pre-requisites.

Strategy: Detect cycle in a directed graph using Topological Sort (Kahn's Algorithm) or DFS coloring.

29. Pacific Atlantic Water Flow (Graphs)

*Complexity: Time: O(R * C), Space: O(R * C)*

Problem: Find cells that can flow to both Pacific and Atlantic oceans.

Strategy: Run DFS/BFS from ocean borders inward. Return cells reachable from both ocean boundaries.

30. Longest Consecutive Sequence (Hashing)

Complexity: Time: O(N), Space: O(N)

Problem: Find longest consecutive elements sequence length.

Strategy: Insert all numbers into a Hash Set. For each number, if (num - 1) is not in set, count consecutive numbers upward.